Parallel & Distributed Computing
Fault Tolerance in Distributed Systems

Farhad M. Riaz

Farhad.Muhammad@numl.edu.pk

Department of Computer Science
NUML, Islamabad

Fundamentals

- What is fault?
  - A fault is a blemish, weakness, or shortcoming of a particular hardware or software component.
  - Fault, error and failures
- Why fault tolerant?
  - Availability, reliability, dependability, …
- How to provide fault tolerance ?
  - Replication
  - Checkpointing and message logging
  - Hybrid

Reliability

- Reliability is an emerging and critical concern in traditional and new settings
  - Transaction processing, mobile applications, cyberphysical systems
- New enhanced technology makes devices vulnerable to errors due to high complexity and high integration
- Technology scaling causes problems
  - Exponential increase of soft error rate
- Mobile/pervasive applications running close to humans
  - E.g Failure of healthcare devices cause serious results
- Redundancy techniques incur high overheads of power and performance
  - TMR (Triple Modular Redundancy) may exceed 200% overheads without optimization [Nieuwland, 06]

Classification of Failures

Crash failure

Omission failure

Transient failure

Software failure

Security failure

Temporal failure

Byzantine failure

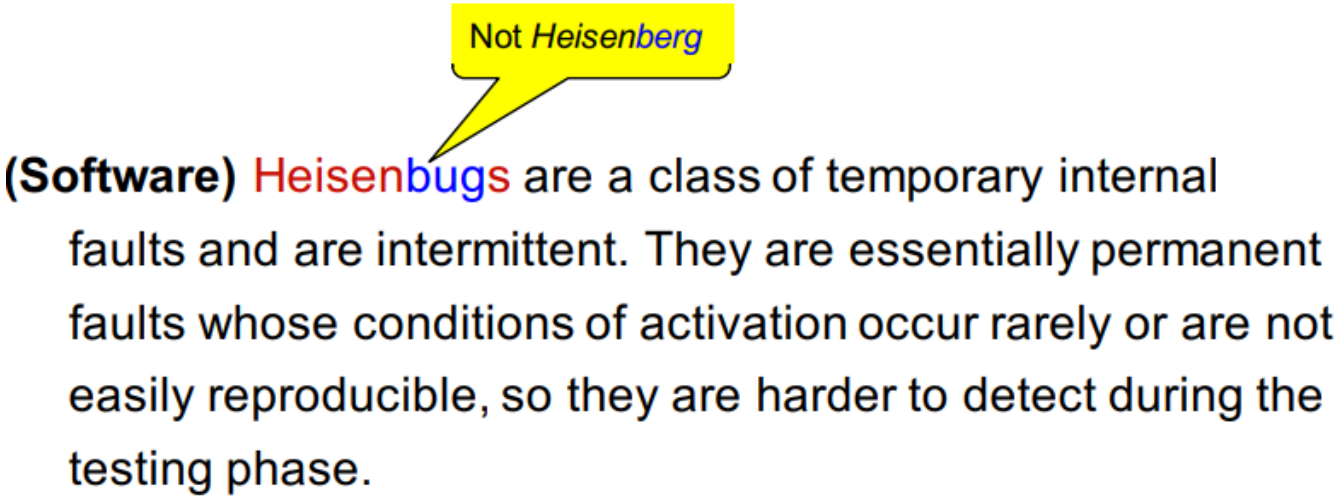
Environmental perturbations

Crash Failures

- Crash failure = the process halts. It is irreversible
- In synchronous system, it is easy to detect crash failure (using heartbeat signals and timeout). But in asynchronous systems, it is never accurate, since it is not possible to distinguish between a process that has crashed, and a process that is running very slowly.
- Some failures may be complex and nasty. Fail-stop failure is a simple abstraction that mimics crash failure when program execution becomes arbitrary. Implementations help detect which processor has failed. If a system cannot tolerate fail-stop failure, then it cannot tolerate crash

Transient Failure

- (Hardware) Arbitrary perturbation of the global state. May be induced by power surge, weak batteries, lightning, radio frequency interferences, cosmic ray



Temporal Failures

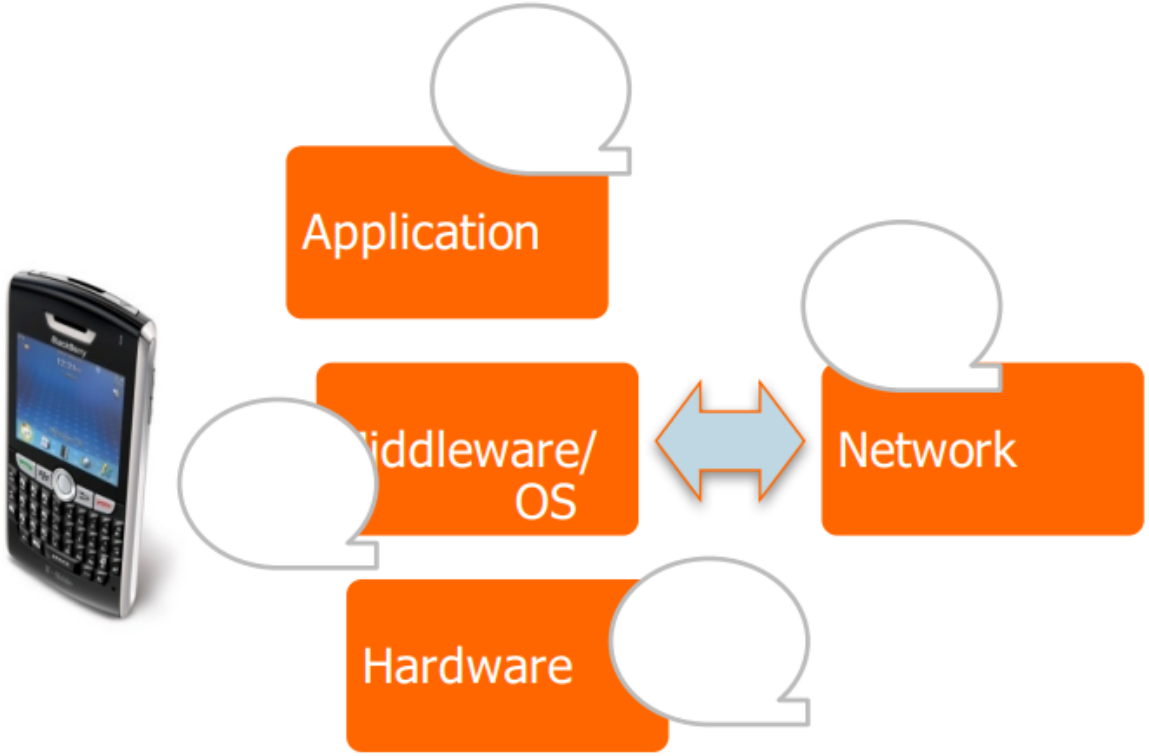
- Inability to meet deadlines
  - Correct results are generated, but too late to be useful. Very important in real-time systems.
- May be caused by poor algorithms, poor design strategy or loss of synchronization among the processor clocks

Byzantine Failure

- Anything goes! Includes every conceivable form of erroneous behavior. The weakest type of failure
- Numerous possible causes. Includes malicious behaviors (like a process executing a different program instead of the specified one) too.
- Most difficult kind of failure to deal with.

Errors/Failures across system layers

- Faults or Errors can cause Failures



Hardware Errors and Error 11 Control Schemes

Failures	Causes	Metrics	Traditional Approaches
Soft Errors, Hard Failures, System Crash	External Radiations, Thermal Effects, Power Loss, Poor Design, Aging	FIT, MTTF, MTBF	Spatial Redundancy (TMR, Duplex, RAID-1 etc.) and Data Redundancy (EDC, ECC, RAID-5, etc.)

- Hardware failures are increasing as technology scales
  - (e.g.) SER increases by up to 1000 times [Mastipuram, 04]
- Redundancy techniques are expensive
  - (e.g.) ECC-based protection in caches can incur 95% performance penalty [Li, 05]

- FIT: Failures in Time (10^9 hours)
- MTTF: Mean Time To Failure
- MTBF: Mean Time b/w Failures
- TMR: Triple Modular Redundancy
- EDC: Error Detection Codes
- ECC: Error Correction Codes
- RAID: Redundant Array of Inexpensive Drives

Software Errors and Error 14 Control Schemes

Failures	Causes	Metrics	Traditional Approaches
Wrong outputs, Infinite loops, Crash	Incomplete Specification, Poor software design, Bugs, Unhandled Exception	Number of Bugs/Klines, QoS, MTTF, MTBF	Spatial Redundancy (N-version Programming, etc.), Temporal Redundancy (Checkpoints and Backward Recovery, etc.)

- Software errors become dominant as system's complexity increases
  - (e.g.) Several bugs per kilo lines
- Hard to debug, and redundancy techniques are expensive
  - (e.g.) Backward recovery with checkpoints is inappropriate for real-time applications

Software Failures

- Coding error or human error
  - On September 23, 1999, NASA lost the $125 million Mars orbiter spacecraft because one engineering team used metric units while another used English units leading to a navigation fiasco, causing it to burn in the atmosphere.
- Design flaws or inaccurate modeling
  - Mars pathfinder mission landed flawlessly on the Martial surface on July 4, 1997. However, later its communication failed due to a design flaw in the real-time embedded software kernel VxWorks. The problem was later diagnosed to be caused due to priority inversion, when a medium priority task could preempt a high priority one.

Software Failures

- Memory leak
  - Processes fail to entirely free up the physical memory that has been allocated to them. This effectively reduces the size of the available physical memory over time. When this becomes smaller than the minimum memory needed to support an application, it crashes.

Network Errors and Error 17 Control Schemes

Failures	Causes	Metrics	Traditional Approaches
Data Losses, Deadline Misses, Node (Link) Failure, System Down	Network Congestion, Noise/Interference, Malicious Attacks	Packet Loss Rate, Deadline Miss Rate, SNR, MTTF, MTBF, MTTR	Resource Reservation, Data Redundancy (CRC, etc.), Temporal Redundancy (Retransmission, etc.), Spatial Redundancy (Replicated Nodes, MIMO, etc.)

- Omission Errors – lost/dropped messages
- Network is unreliable (especially, wireless networks)
  - Buffer overflow, Collisions at the MAC layer, Receiver out of range
- Joint approaches across OSI layers have been investigated for minimal costs [Vuran, 06][Schaar, 07]

•SNR: Signal to Noise Ratio
•MTTR: Mean Time To Recovery
•CRC: Cyclic Redundancy Check
•MIMO: Multiple-In Multiple-Out

Classifying Fault-Tolerance

- Masking tolerance.
  - Application runs as it is. The failure does not have a visible impact. All properties (both liveness & safety) continue to hold.
  - Non-masking tolerance.
    - Safety property is temporarily affected, but not liveness.
  - Example 1.
    - Clocks lose synchronization, but recover soon thereafter.
  - Example 2.
    - Multiple processes temporarily enter their critical sections, but thereafter, the normal behavior is restored

Classifying Fault-Tolerance

- Fail-safe tolerance
  - Given safety predicate is preserved, but liveness may be affected
  - Example.
    - Due to failure, no process can enter its critical section for an indefinite period. In a traffic crossing, failure changes the traffic in both directions to red.
- Graceful degradation
  - Application continues, but in a “degraded ”mode. Much depends on what kind of degradation is acceptable.
  - Example. Consider message-based mutual exclusion. Processes will enter their critical sections, but not in timestamp order

Conventional Approaches

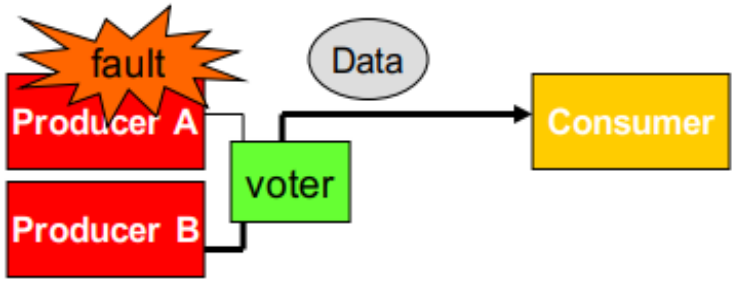
- Build redundancy into hardware/software
  - Modular Redundancy, N-Version Programming Conventional TRM (Triple Modular Redundancy) can incur 200% overheads without optimization. I
  - Replication of tasks and processes may result in overprovisioning
  - Error Control Coding
- Checkpointing and rollbacks
  - Usually accomplished through logging (e.g. messages)
  - Backward Recovery with Checkpoints cannot guarantee the completion time of a task.
- Hybrid
  - Recovery Blocks

Modular Redundancy

- Modular Redundancy
  - Multiple identical replicas of hardware modules
  - Voter mechanism
    - Compare outputs and select the correct output

➔Tolerate most hardware faults

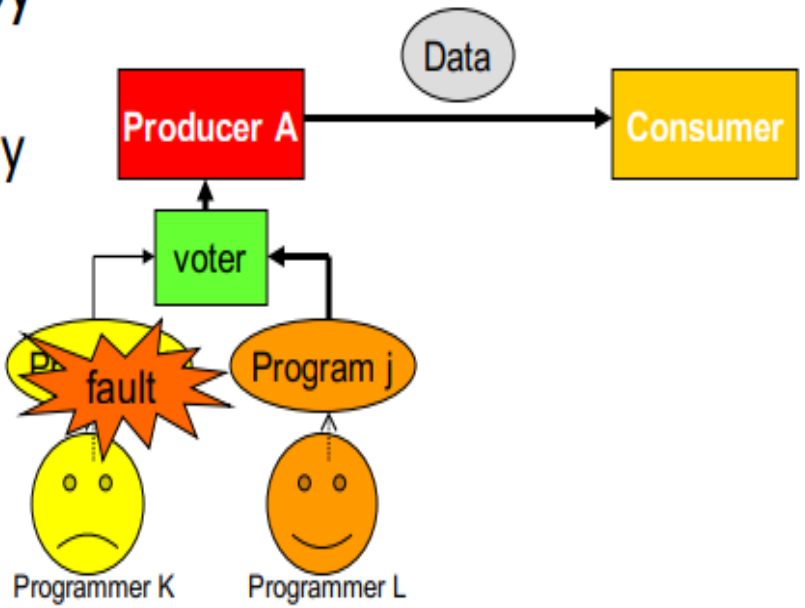
➔Effective but expensive



N-version Programming

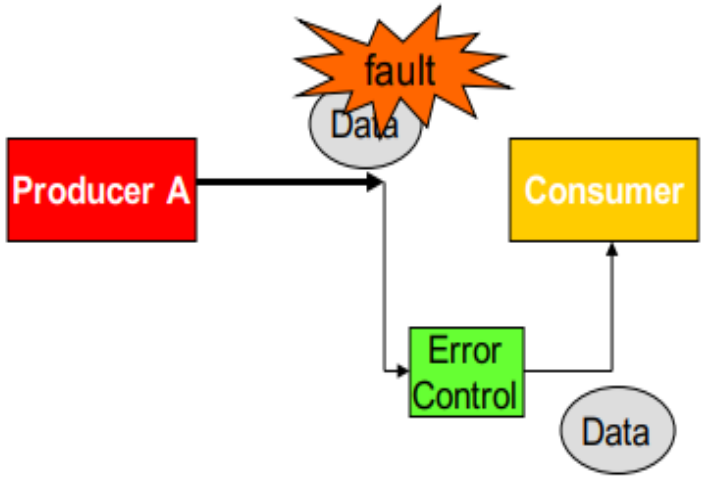
- N-version Programming
  - Different versions by different teams
    - Different versions may not contain the same bugs
  - Voter mechanism

➔Tolerate some software bugs



Error-Control Coding

- Error-Control Coding
  - Replication is effective but expensive
  - Error-Detection Coding and Error-Correction Coding
    - (example) Parity Bit, Hamming Code, CRC
  - ➔ Much less redundancy than replication



Consensus

- Reaching Agreement is a fundamental problem in distributed computing
- Mutual Exclusion
  - processes agree on which process can enter the critical section
- Leader Election
  - the processes agree on which is the elected process
- Totally Ordered Multicast
  - the processes agree on the order of message delivery
- Commit or Abort in distributed transactions
  - Reaching agreement about which process has failed
- Other examples
  - Air traffic control system: all aircrafts must have the same view
  - Spaceship engine control – action from multiple control processes(“proceed” or “abort”)
  - Two armies should decide consistently to attack or retreat

Defining Consensus

- N processes
- Each process p has
  - input variable xp : initially either 0 or 1 1.
  - output variable yp : initially b (b=undecided) – can be changed only once
- Consensus problem:
  - design a protocol so that either
    - all non-faulty processes set their output variables to 0
    - Or non-faulty all processes set their output variables to 1
    - There is at least one initial state that leads to each outcomes 1 and 2 above

Solving Consensus

- No failures -trivial
  - All-to-all broadcast
  - With failures
    - Assumption: Processes fail only by crash-stopping
  - Synchronous system:
    - bounds on I Message delays
    - Max time for each process step
    - e.g., multiprocessor (common clock across processors)
  - Asynchronous system:
  - no such bounds! e.g., The Internet! The Web!

Variant of Consensus Problem

- Consensus Problem (C)
  - Each process propose a value
  - All processes agree on a single value
- Byzantine General Problem (BG)
  - Process fails arbitrarily, byzantine failure
  - Still processes need to agree
- Interactive Consistency (IC)
  - Each process propose its value
  - All processes agree on the vector

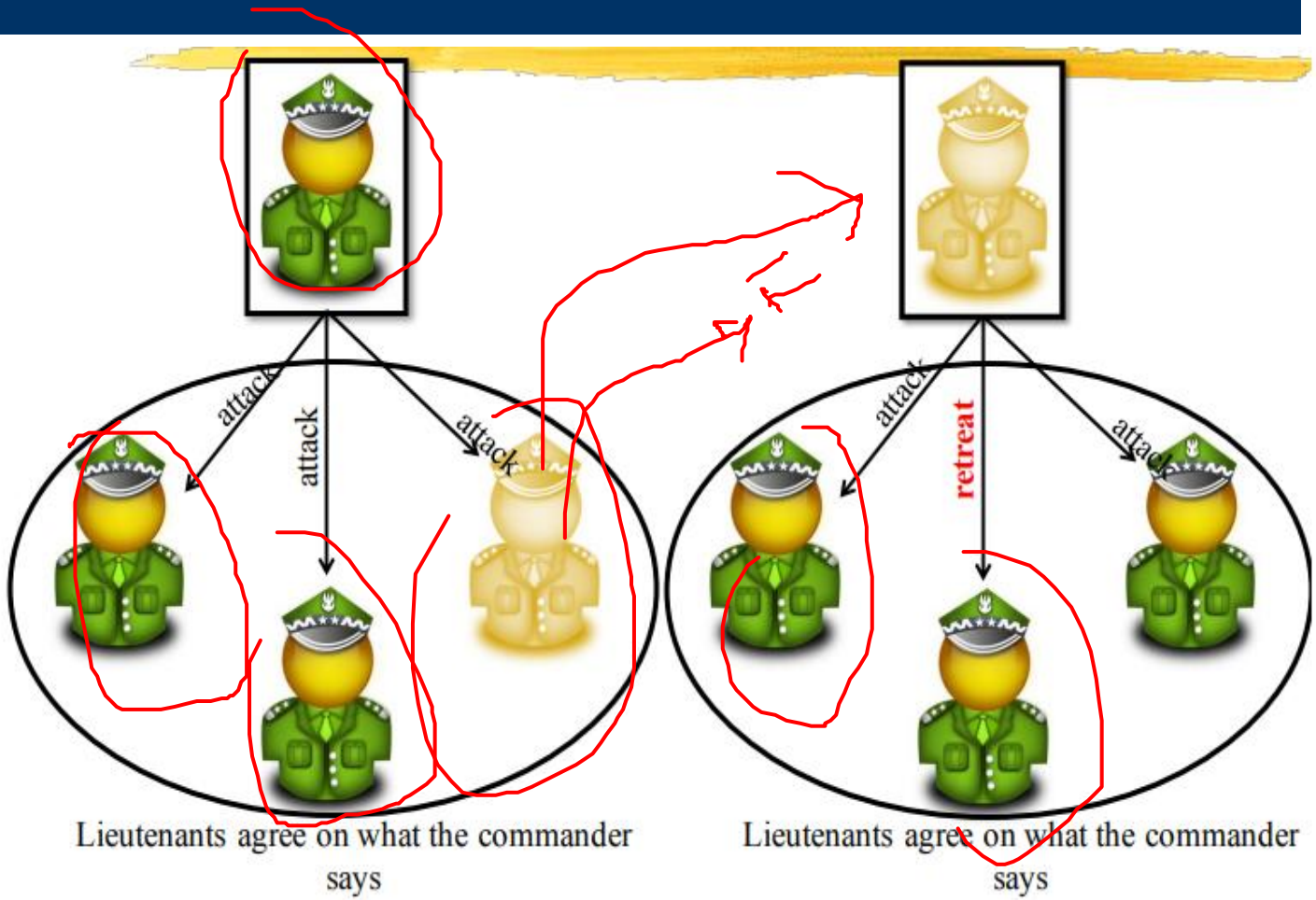
Consensus in Synchronous System

- Possible
  - With one or more faulty processes
- Solution:
  - Basic idea: all processes exchange (multicast) what other processes tell them in several rounds
- Proof:
  - to reach consensus with f failures, the algorithm needs to run in f + 1 rounds
  - Basic idea: if A and B do not agree on value X, then some other process C, did send X to A, but not to B.
  - Requires 1 failure in each round, so f+1 fails.
  - Contradiction

Asynchronous Consensus

- Messages have arbitrary delay, processes arbitrarily slow
- Impossible to achieve!
  - a slow process indistinguishable from a crashed process
- Theorem:
  - In a purely asynchronous distributed system, the consensus problem is impossible to solve if even a single process crashes
- Result due to Fischer, Lynch, Patterson (commonly known as FLP 85).

Byzantine General Problem



Byzantine General Problem

The Byzantine generals problem • In the informal statement of the *Byzantine generals problem* [Lamport *et al.* 1982], three or more generals are to agree to attack or to retreat. One, the commander, issues the order. The others, lieutenants to the commander, must decide whether to attack or retreat. But one or more of the generals may be 'treacherous' – that is, faulty. If the commander is treacherous, he proposes attacking to one general and retreating to another. If a lieutenant is treacherous, he tells one of his peers that the commander told him to attack and another that they are to retreat.

The Byzantine generals problem differs from consensus in that a distinguished process supplies a value that the others are to agree upon, instead of each of them proposing a value. The requirements are:

Termination: Eventually each correct process sets its decision variable.

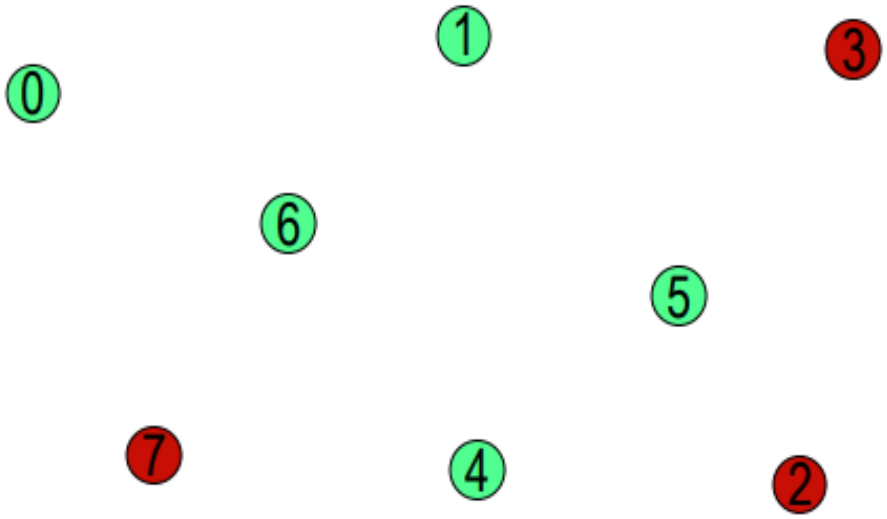
Agreement: The decision value of all correct processes is the same: if p_i and p_j are correct and have entered the *decided* state, then $d_i = d_j$ $(i, j = 1, 2, \ldots, N)$.

Integrity: If the commander is correct, then all correct processes decide on the value that the commander proposed.

Failure Detection

- The design of fault-tolerant algorithms will be simple if processes can detect failures.
- In synchronous systems with bounded delay channels, crash failures can definitely be detected using timeouts.
- In asynchronous distributed systems, the detection of crash failures is imperfect.
- Completeness – Every crashed process is suspected
- Accuracy – No correct process is suspected.

Example



0 suspects {1,2,3,7} to have failed. Does this satisfy completeness?
Does this satisfy accuracy?

Classification of Completeness

- Strong completeness.
  - Every crashed process is eventually suspected by every correct process, and remains a suspect thereafter.
- Weak completeness.
  - Every crashed process is eventually suspected by at least one correct process, and remains a suspect thereafter.
  - Note that we don't care what mechanism is used for suspecting a process

Classification of accuracy

- Strong accuracy.
  - No correct process is ever suspected.
- Weak accuracy.
  - There is at least one correct process that is never suspected.

Eventual Accuracy

- A failure detector is eventually strongly accurate, if there exists a time T after which no correct process is suspected.
- (Before that time, a correct process be added to and removed from the list of suspects any number of times)
- A failure detector is eventually weakly accurate, if there exists a time T after which at least one process is no more suspected

Classifying Failure Detectors

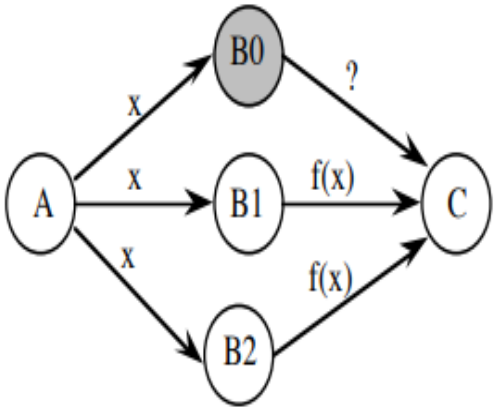
- Perfect P. (Strongly) Complete and strongly accurate
- Strong S. (Strongly) Complete and weakly accurate
- Eventually perfect ◊P. (Strongly) Complete and eventually strongly accurate
- Eventually strong ◊S (Strongly) Complete and eventually weakly accurate
- Other classes are feasible: W (weak completeness) and weak accuracy) and ◊W

Detection of Crash Failures

- Failure can be detected using heartbeat messages (periodic “I am alive” broadcast) and timeout
  - if processors speed has a known lower bound
  - channel delays have a known upper bound

Tolerating Crash Failures

Triple modular redundancy (TMR) for masking any single failure. *N-modular redundancy masks up to m failures, when N = 2m +1.*



Take a vote

What if the voting unit fails?

Detection of Omission Failures

- For FIFO channels: Use sequence numbers with messages.
  - (1, 2, 3, 5, 6 …) ⇒ message 4 is missing
  - Non-FIFO bounded delay channels- use timeout
  - What about non-FIFO channels for which the upper bound of the delay is not known?
  - Use unbounded sequence numbers and acknowledgments. But acknowledgments may be lost too!
  - Let us look how a real protocol deals with omission …

Tolerating Omission Failures

A central issue in networking

Routers may drop messages, but reliable end-to-end transmission is an important requirement. If the sender does not receive an ack within a time period, it retransmits (it may so happen that the was not lost, so a duplicate is generated). This implies, the communication must tolerate **Loss, Duplication, and Re-ordering** of messages